Tarea 2: Torneo de Komikan concurrente

Roberto Ibaceta · Gabriel Varas · Entrega: 3/11/2026

1. Diseño de sincronización

Todo corre en un solo proceso con hilos POSIX. Hay tres niveles: el servidor (clientes, cola de espera y límite de mesas), cada mesa (un juez y 13 fichas) y el manejo de SIGINT.

Servidor

- **Llegada y cola.** Cada cliente es un hilo que espera su retardo de llegada con `pthread_cond_timedwait` sobre la condición del servidor, de modo que un SIGINT corta la espera de inmediato. Al llegar se encola (arreglo FIFO protegido por `mx`) y hace `broadcast`.
- **Emparejador.** El hilo principal espera con `pthread_cond_wait` hasta que haya dos clientes en la cola o hayan llegado todos. Saca dos en orden de llegada: el primero es Puma y el segundo Perros. Si N es impar, el último se retira sin jugar.
- **Límite de M mesas.** Un semáforo POSIX iniciado en M. El emparejador hace `sem_wait` (bloqueante, sin espera activa y sin tener `mx` tomado) antes de lanzar el hilo de la partida, y ese hilo hace `sem_post` al terminar y destruir su mesa. Usamos `sem_open` con nombre y `sem_unlink` inmediato porque macOS no implementa `sem_init`.

Mesa

- **Un mutex y una condición por mesa** protegen todo su estado: casillas, posiciones, estado de cada ficha, la solicitud de movimiento en curso y el contador de capturas.
- **Fichas.** Cada ficha repite: dormir un tiempo aleatorio entre `min_ms` y `max_ms` (con `timedwait`, para despertar si la partida termina), esperar a que el tablero esté libre, tomarlo, elegir un movimiento, publicar la solicitud y esperar a que el juez la resuelva. No hay orden fijo: toma el tablero la primera ficha que despierta.
- **Juez.** Espera solicitudes, valida adyacencia o captura (salto en línea recta sobre un perro hacia una casilla vacía), aplica el movimiento, lo registra en `logs/mesa_<id>.log` y revisa el fin: 6 capturas gana el Puma, Puma sin movimientos ganan los Perros, 2000 movimientos es empate.
- **Misma casilla.** La solicitud y su validación ocurren bajo el mismo mutex y el juez exige que el destino esté vacío. Dos fichas nunca pueden quedar con la misma casilla: la segunda ya la ve ocupada.

Fallos y SIGINT

- **Fallo de un jugador.** En cada ciclo una ficha puede fallar con probabilidad `prob_fallo`. Si falla, se marca como caída y termina con `pthread_exit` sin devolver el tablero, simulando un jugador que muere con el recurso tomado. El juez lo detecta, recupera el tablero, da la victoria por abandono al rival, marca la partida como terminada, despierta a todas las fichas y hace `join` de las 13 antes de destruir el mutex y la condición. Un movimiento inválido también cuenta como abandono.
- **SIGINT.** `main` bloquea SIGINT antes de crear cualquier hilo, así todos heredan la máscara. Un hilo dedicado lo atiende con `sigwait`: pone `apagando = 1` (atómico), despierta la cola, llama `mesa_abortar` en cada mesa activa y hace un `sem_post` extra para despertar al emparejador. Después se hace `join` de todos los hilos y se libera todo; el programa sale con código 130.

2. Deadlock, livelock y starvation

Deadlock

Dentro de una mesa cada hilo usa un único mutex, así que nunca retiene uno mientras espera otro: no se cumple la condición de retención y espera, y no puede haber espera circular. Las únicas esperas son `cond_wait` con predicado, y todo cambio de estado hace `broadcast`; el fin de la partida pone `terminada = 1` y despierta a todos.

El tablero es un recurso lógico. Si una ficha muere teniéndolo, las demás se quedarían esperando para siempre: es exactamente el caso de fallo, y por eso el juez lo recupera. Como la ficha suelta el mutex antes de terminar, no hacen falta mutex robustos (`EOWNERDEAD`).

Entre el servidor y las mesas el orden de adquisición es siempre el mismo: `servidor_apagar` toma el mutex del servidor y luego el de la mesa, y ningún hilo de mesa toma el del servidor teniendo el suyo. El hilo de la partida saca su mesa de la lista de activas (con el mutex del servidor) antes de destruirla, así el apagado nunca aborta una mesa ya liberada. El `sem_wait` se hace sin mutex tomado.

Livelock

Ninguna ficha reintenta en un bucle cerrado. Si no tiene movimiento, suelta el tablero y vuelve a dormir un tiempo aleatorio, lo que actúa como *backoff*. El juez no pide reintentos: un movimiento inválido termina la partida. El riesgo restante es una partida que avanza pero nunca termina (puma y perros moviéndose sin resultado). Para eso está el tope de 2000 movimientos, que la cierra como empate.

Starvation

El mutex de pthreads no es FIFO, así que en teoría una ficha podría perder siempre la carrera por el tablero. En la práctica la ficha que acaba de mover duerme entre 50 y 300 ms antes de volver a competir, y la probabilidad de perder una y otra vez cae geométricamente. No es una garantía estricta: para eso habría que repartir el tablero con una cola FIFO o con tickets. En el servidor no hay inanición: la cola es FIFO y solo el emparejador hace `sem_wait`, así que los pares juegan en orden de llegada.

3. Trade-offs

Decisión	A favor	En contra
Un mutex por mesa en vez de uno por casilla	Validar una captura mira tres casillas y además el estado global (capturas, si el puma quedó encerrado). Con un solo lock eso es atómico y el análisis de deadlock es trivial.	Los movimientos de una misma mesa se serializan. Aplicar uno toma microsegundos frente a retardos de 50–300 ms, así que lo que se pierde es mínimo; el paralelismo real está entre las M mesas, cada una con su propio lock.
El juez valida cada movimiento	Lo pide el enunciado y deja las reglas en un solo lugar; una ficha defectuosa no puede corromper el tablero.	Dos cambios de contexto extra por movimiento (ficha → juez → ficha).
Juez con `timedwait` de 100 ms	Respaldo para ver `apagando` aunque se pierda un aviso. La latencia de aborto medida fue de 1 a 13 ms.	Despierta hasta 10 veces por segundo sin trabajo; costo despreciable.
Un hilo por ficha	Lo exige el enunciado y modela bien jugadores independientes.	14 hilos por mesa; con M grande son muchos hilos casi siempre dormidos.
Semáforo con nombre	Funciona igual en Linux y macOS; el `unlink` inmediato no deja nada en el sistema.	Algo más de código que `sem_init`.

Verificación

Compila sin advertencias con `-Wall -Wextra -std=c17 -pthread`. `make test` cubre N impar, N=1, M=1, N=40 con M=4 (nunca más de 4 mesas simultáneas), argumentos y configuraciones inválidas, SIGINT a mitad de torneo (cierre en menos de 1 s), fallos simulados, ThreadSanitizer sin advertencias y valgrind sin fugas, tanto en una corrida normal como interrumpida con SIGINT.